Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Bu kayıt bir uygulama dersidir. Hoca Göksel Biricik oturumu açıp asistana devretti (00:01:58).
  - Asistanın adı kayıtta net anlaşılmıyor. Altyazıda “Atıcı Hocam” diye geçiyor.
- Örnek proje: Spor Salonu Sistemi
  - Problem tanımı ve aktörler.
  - Fizibilite çalışması: altı fizibilite türü ve fizibilite matrisi.
  - Gereksinim analizi: anket, kullanım senaryosu (use case) dokümanı ve diyagramı, veri akış diyagramı (DFD), veri sözlüğü, varlık-ilişki diyagramı (ERD).
  - Sistem tasarımı: yapı diyagramı, sınıf diyagramı, ardışıl (sequence) diyagram.
  - Değerlendirme tablosu ve özet.

Hocanın Özellikle Vurguladığı Kısımlar

- Sadece örneğe odaklanmayın; ER, use case ve veri akış gibi diyagramlar önemlidir (00:46:44).
- Veri akış diyagramında her işlemin mutlaka bir girdisi ve bir çıktısı olmalı.
- Fizibilite matrisinde tek sistem değil, birden çok alternatif önerin. Karşılaştırma ancak böyle yapılır.
- Yazılım geliştirme modelini projeye göre doğru seçin.
  - Şelale modelinde geri besleme yoktur. Daha önce benzerini yapmadığınız bir projede uygun değildir.
- Gantt planı sarkarsa ara toplantıda belirtin ve planı güncelleyin.
  - MS Project, bir işin süresi değişince sonraki işlerin tarihlerini otomatik kaydırır.
- Kişisel veriler KVKK kapsamında korunmalı ve bu üyelik sırasında kullanıcıya bildirilmeli.

Kısa Tekrar Notları

- **Aktörler:**
  - Spor salonu yönetimi
  - Üye
  - Eğitmen
- **Fizibilite türleri (6):**
  - Teknik
  - Ekonomik
  - Zaman
  - Sosyal
  - Yönetim
  - Yasal
- **Fizibilite matrisi (slayttan):**

	Sistem 1	Sistem 2	Sistem 3
Teknik	Java, JavaScript, 1 server	C#, .NET, 2 server	Python, Django, cloud, güvenlik hizmeti
Ekonomik	20.300 USD	25.300 USD	35.000 USD
Zaman	27 gün	30 gün	25 gün
Sosyal	Kabul edilebilir	Kabul edilebilir	Kabul edilebilir
Yönetim	Sıkıntı yok	Sıkıntı yok	Sıkıntı yok
Yasal	KVKK	KVKK	KVKK + güvenlik hizmeti

  - Örnekte maliyeti en düşük olan Sistem 1 seçildi.

- **Use case diyagramı:**
  - Aktör çöp adamla, senaryo elipsle, sistem sınırı dikdörtgenle gösterilir.
  - **include:** Bir senaryo diğerini her zaman içerir. Örnek: Sınıfa kaydolma, sınıf bilgisini görüntülemeyi içerir.
  - **extend:** İsteğe bağlı genişletmedir. Örnek: Özel ders talebi, sınıfa kaydolmayı genişletir.
- **DFD seviyeleri:**
  - Seviye 0: Kaba bağlam diyagramı.
  - Seviye 1: İşlemler ve veri depoları açılır.
  - Seviye 2: Bir işlem (örnek: ödeme) alt işlemlerine ayrılır.
- **ERD (Chen gösterimi):**
  - Varlık dikdörtgenle, nitelik elipsle, ilişki eşkenar dörtgenle gösterilir.
  - İlişkilere de nitelik eklenebilir. Örnek: Kaydolma ilişkisinin kayıt tarihi.
- **Değerlendirme tablosu (100 üzerinden):** Fizibilite çalışması toplam 30 puan, gereksinim analizi 40 puan, sistem tasarımı 30 puan.
  - Fizibilite puanlarının dağılımı: teknik 8, ekonomik 7, zaman 7, sosyal 2, yönetim 3, yasal 3.

Detaylı Açıklamalar

Problem tanımı

- Hizmet veren bir spor salonunun müşteri sayısı ve ders çeşitliliği arttı.
  - Mevcut sistemde kayıt salona gidilerek yapılıyor, kayıtlar yerelde tutuluyor. Bu durum zaman kaybına yol açıyor.
- **Yeni sistem çevrim içi olacak:**
  - Üyelik çevrim içi yapılacak.
  - Sınıf takvimi ve eğitmen bilgileri görüntülenebilecek.
  - Aerobik, step, pilates, dans gibi özel derslere kayıt yapılabilecek.
- **Ücretlendirme:**
  - Her ay sabit bir üyelik ücreti alınır; bu ücret fitness salonunun süresiz kullanımını kapsar.
  - Özel eğitimler (grup veya birebir) için ayrıca ücret ödenir.
- **Yönetim:**
  - Üyelik başvurularını onaylar.
  - Sınıf oluşturur; tarih, saat ve eğitmen bilgisini girer ve günceller.
- **Üye:**
  - Üye olur ve takvime bakıp kontenjanı olan sınıfa kaydolur.
  - İsterse özel ders talep eder.
- **Eğitmen:**
  - Bilgilerini günceller.
  - Ders verdiği sınıfların katılımcı listesini görür.

Fizibilite çalışması

- **Teknik fizibilite:** Gerekli yazılım ve donanım araştırılır.
  - **Yazılım:** Örnekte nesneye dayalı bir dil seçildi; Java ve JavaScript ile ön ve arka uç geliştirileceği varsayıldı. C#, Python/Django gibi alternatifler de olabilir.
  - **Donanım:** Spor salonunun bilgisayarları eski. Yeni masaüstü bilgisayar, server ve wireless router gerekiyor. Donanımların teknik özellikleri yazılır, çünkü ekonomik fizibilite bunlara göre hesaplanır.
  - **Ölçeklenme:** Kullanıcı sayısı artınca veri tabanı için ayrı sunucu alınabilir veya sistem buluta taşınabilir. Bu da ayrı maliyet getirir.
- **Ekonomik fizibilite:** Yazılım şirketine ödenecek ücret ve bilgisayar, router, server, yazılım araçları gibi masraflar çıkarılır.
  - Projenin başa baş (kâra geçiş) noktası tahmin edilir. Slayttaki tutarlar semboliktir.
- **Zaman fizibilitesi:** Sistemin ne kadar sürede planlanıp hayata geçeceği belirlenir.
  - Gantt çizelgesiyle hangi işi hangi kaynağın yapacağı ve saatlik ücretleri planlanır. PERT diyagramı da kullanılabilir.
  - Yazılım geliştirme modeli zamanı etkiler. Örnekte şelale modeli seçildi; alternatif olarak yinelemeli (iteratif) şelale önerildi.
    - Yinelemeli şelalede gerçekleştirme sırasında tasarım hatası görülürse tasarıma dönülebilir.
- **Sosyal fizibilite:** Sistem toplumda kabul görecek mi? Farklı spor salonları bu tür sistemleri zaten kullanıyor.
  - Gençler spora ve internete yatkın, çevrim içi işlem yapan yaş aralığı genişledi. Sonuç olarak sistem kabul görebilir.
- **Yönetim fizibilitesi:** Projenin yönetim tarafından istenmesi ve desteklenmesi gerekir. Yönetimin desteklemediği proje zor hayata geçer.
- **Yasal fizibilite:** Kişisel veriler KVKK gereği korunur. Bu durum üyelik sırasında kullanıcıya bildirilir.
- **Fizibilite matrisi:** Üç sistem karşılaştırıldı.
  - Sistem 3 bulut ve güvenlik hizmetiyle avantajlı görünüyor ama en pahalısı.
  - Sistem 2 iki sunucu kullanıyor, başlangıç için pahalı.
  - Başlangıçta tek sunucu yeterli olduğu için Sistem 1 seçildi.
- **Nesneye yönelik dil seçiminin gerekçeleri (slayttan):**
  - Değişiklik tek bir sınıfta yapılabilir.
  - Sınıflar sayesinde kod tekrarı azalır.
  - Bilgi gizleme sağlanır.

Gereksinim analizi

- Ön inceleme ve fizibilite raporları incelendikten sonra ayrıntılı görüşmelerle gereksinimler belirlenir. Tasarımın temeli burada atılır.
- **Veri toplama:** Kullanıcılara açık ve kapalı uçlu sorulardan oluşan bir anket verildi. Örnek soru: “Var olan sistemde sorun yaşıyor musunuz?”
- **Use case dokümanı (ödeme senaryosu):**
  - **Birincil aktör:** Üye.
  - **İlgiler:** Üye, doğru hesaplanan ücreti sorunsuz ödemek ister.
  - **Ön koşul:** Kişi üye olmalı veya sistemde çevrim içi bulunmalı (00:26:03).
  - **Ana akış:**
    1. Üye ödenecek tutarı görüntüler.
    2. Kredi kartı ve fatura bilgilerini girer.
    3. Üyenin onayıyla ödeme gerçekleşir.
  - **Alternatif akış:** Bilgi yanlışsa sistem hata mesajı verir ve bilgilerin yeniden girilmesini ister.
  - Her senaryo için böyle bir doküman hazırlanır.
- **Use case diyagramı:**
  - **Üye:** Üye olur, sınıf bilgisini görüntüler, sınıfa kaydolur (include: sınıf bilgisini görüntüle), özel ders seçer (extend), ödeme yapar.
  - **Eğitmen:** Katılımcıları görür, bilgilerini günceller.
  - **Yönetim:** Sınıf oluşturur.
- **Veri akış diyagramı:**

  - **Seviye 0:** Spor salonu sistemi tek kutu; üç aktörle veri alışverişi gösterilir.
    * Üye kişisel bilgilerini ve sınıf seçimini gönderir, sınıf bilgisini alır, ücret öder.
  - **Seviye 1:** İşlemler açılır.
    * Üye olma işlemi kişisel bilgileri alır, "Üyeler" veri deposuna yazar.
    * Sınıf oluşturma işlemi yönetimden sınıf bilgisini alır, "Sınıf bilgisi" deposuna yazar.
  - **Seviye 2:** Ödeme yapma alt işlemlere ayrılır: toplam ücretin hesaplanması, ödemenin alınması, kalan borcun hesaplanması.
- **Veri sözlüğü:** Her veri akışı için ad, içerik, kaynak, hedef ve miktar yazılır.
  - **Örnek 1, kişisel bilgiler:**
    * İçerik: ad, soyad, yaş, boy, kilo, iletişim.
    * Veri yapıları: üye detayları (00:32:29).
    * Kaynak üye, hedef "üye olma" işlemi.
  - **Örnek 2, sınıf bilgileri:**
    * İçerik: ders, eğitmen, katılımcılar, takvim.
    * Miktar: günde ortalama 10 giriş.
- **Varlık-ilişki diyagramı:**
  - **Varlıklar:**
    * Üye: ad-soyad, yaş, boy, kilo, iletişim.
    * Sınıf: isim, yer, tarih, saat, kapasite.
  - **İlişkiler:**
    * Üye-Sınıf: kaydolma (nitelik: kayıt tarihi).
    * Üye-Muhasebe: ödeme yapma.
    * Muhasebe-Yönetim: raporlama.
    * Yönetim-Sınıf: sınıf oluşturma.
    * Sınıf-Eğitmen: ders verme.

Sistem tasarımı

- **Yapı diyagramı (structure chart):** Ana modül (spor salonu) ve alt modüller (üye ol, giriş yap, sınıfa kaydol…) arasındaki veri akışını hiyerarşik gösterir.
  - Oklardaki içi boş ve içi dolu yuvarlaklar farklı anlam taşır. Asistana göre dolu yuvarlak onay ve doğrulama mesajlarında kullanılır.
  - Eşkenar dörtgen bir seçimi gösterir. Sınıfa kaydolan üye grup dersine veya özel derse katılabilir.
- **Sınıf diyagramı (kareden):**
  - **Sınıflar:**
    * Spor Salonu Yönetimi: `borcOgren()`.
    * Üye: `uyeNo`, `isim`, `yas`, `boy`, `kilo`.
    * Üyelik: `uyeOl(Uye)`.
    * Sınıf: `id`, `ders`, `takvim`, `egitmen`, `uyeler: List<Uye>`, `getSinifBilgisi()`.
    * Eğitmen: `id`, `isim`, `branslar: String[]`.
    * Soyut `Ders`: `isim`, `saatUcreti: double`, `getSaatUcreti()`.
  - **Kalıtım:** `Aerobik`, `Dans` ve `Pilates`, soyut `Ders` sınıfından kalıtım alır. Soyut sınıftan doğrudan nesne oluşturulamaz.
  - **İlişkiler:**
    * Bir eğitmen birden çok sınıfa (`1..*`) ders verir.
    * Bir üyenin bir üyeliği vardır (1-1).
    * Üye ile Sınıf arasında içi boş elmaslı ilişki var.
- **Ardışıl (sequence) diyagram:**
  - Üye nesnesi `uyeOl(uyelikBilgileri)` çağırır, spor salonu onay mesajı döner. Giden mesajlar düz, dönüşler kesikli çizgidir.
  - Sınıf bilgisi alma ve sınıfa kaydolma çağrıları Sınıf nesnesine gider.
  - **Borç öğrenme:**
    1. Üye spor salonuna `borcOgren` mesajı gönderir.
    2. Spor salonu Sınıf'tan ödenecek tutarı ister.
    3. Sınıf, Ders'ten saat ücretini alır.
    4. Tutar, saat ücreti × katılım süresi olarak hesaplanır ve geri döndürülür.
- **Diyagram araçları:** Gantt ve PERT için MS Project kullanılabilir. UML diyagramları için çevrim içi ve masaüstü araçlar var.

Değerlendirme

- Asistan, projede yapılan adımları önemine göre 100 üzerinden puanladı:
  - Fizibilite çalışması 30 puan (teknik 8, ekonomik 7, zaman 7, sosyal 2, yönetim 3, yasal 3).
  - Gereksinim analizi 40 puan.
  - Sistem tasarımı 30 puan.
- Puanlar projenin hedefine, büyüklüğüne ve hitap ettiği kitleye göre değişir.
  - Örnek: Bütçesi bol bir projede teknik fizibilite (daha güçlü donanım, güvenlik hizmeti) öne çıkabilir.

Ders sonrası notlar

- **Puan dağılımı:** Sözlü anlatımda "yasal 3, yönetim ve sosyal 2 gibi" dendi. Slayttaki tabloda sosyal 2, yönetim 3, yasal 3. Toplam $8 + 7 + 7 + 2 + 3 + 3 + 40 + 30 = 100$, tablo tutarlı.
- **Matristeki seçim:** Sistem 3 en pahalı ama en kısa süreli (25 gün). Seçim yalnızca maliyete göre yapıldı. Raporda bu tür bir ödünleşimin (maliyet-süre) gerekçesini açıkça yazmak gerekir.
- **PERT ve Gantt:** Derste Gantt'ın iş adımlarının ilişkilerini PERT'ten daha net gösterdiği söylendi.
  - Gantt çizelgesi zaman çizelgesini ve süreleri iyi gösterir.
  - İşler arasındaki bağımlılıkları ve kritik yolu PERT/CPM ağ diyagramı daha açık gösterir.
- **ERD terimi:** Derste ER diyagramındaki "Üye" için "sınıf" ve "attribute" denildi. ER modelinde bunlar varlık (entity) ve niteliktir (attribute). "Sınıf" UML sınıf diyagramının terimidir.
- **"Zayıf varlık ilişkisi":** Asistan, Üye-Sınıf ilişkisini "zayıf varlık ilişkisi; üye varsa bir sınıfa kayıtlı olmalı" diye açıkladı.
  - Sınıf diyagramındaki içi boş elmas UML'de birleştirme (aggregation) ilişkisidir.
  - "Her üye en az bir sınıfa kayıtlı olmalı" kuralı bir katılım kısıtıdır (zorunlu katılım).
  - Zayıf varlık ise ER modelinde kendi anahtarı olmayan ve sahip varlığa bağlı tanımlanan varlıktır. Bu üç kavram farklıdır.
- **Use case son koşulu:** Ödeme senaryosunda son koşul "ödeme bilgileri doğru olmalıdır" diye yazılmış. Bu bir ön koşul veya doğrulama kuralıdır.
  - Son koşul, senaryo başarıyla bitince sistemin durumunu anlatır. Örnek: "Ödeme kaydedilir, üyenin borcu güncellenir, makbuz üretilir."
- **Ardışıl diyagramdaki dikdörtgenler:** Bunların UML'deki adı aktivasyon (execution occurrence) çubuğudur. Nesnenin bir işlemi yürüttüğü süreyi gösterir.
- **Yapı diyagramı simgeleri:**
  - İçi boş yuvarlaklı ok veri çiftidir (data couple): modüller arasında taşınan veri.
  - İçi dolu yuvarlaklı ok kontrol bayrağıdır (control flag): onay, hata, "bulundu" gibi durum bilgisi.
  - Eşkenar dörtgen koşullu çağrıyı gösterir.
- **"Bilgi gizliliği":** Slayttaki "nesneler birbirinden bağımsız olduğu için bilgi gizliliği sağlar" ifadesinin doğru terimi bilgi gizlemedir (information hiding).
  - Bu, sınıfın iç ayrıntılarının erişim belirleyicileriyle (`private` gibi) dışarıdan saklanmasıdır.
  - Veri güvenliği veya mahremiyet anlamında bir gizlilik değildir.
- **Şelale modeli ve ölçek:** Asistan şelalenin büyük ölçekli projelere uygun olmadığını söyledi. Belirleyici olan yalnızca ölçek değildir.
  - Şelale, gereksinimleri baştan net ve kararlı olan projeler için uygundur.
  - Gereksinimleri belirsiz veya değişken projelerde yinelemeli ve çevik modeller daha uygundur.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.